

Obligatory 1 Report

Emotion Valence Regression

Noor Fatima

Student ID: 2200

Email: noorfa@uio.no

Repository: <https://github.uio.no/noorfa/IN5550-Oblig1>



Introduction

This report presents my solution to IN5550 Oblig 1: supervised emotion valence regression. The task is to predict the normalized valence score in $[0,1]$ from English text.

I implement and compare two feed-forward approaches: (1) a Bag-of-Words baseline and (2) static word-embedding models (pre-trained and own-trained).

Experiments are run on a fixed train/dev split and evaluated with MSE, MAE, RMSE, Pearson, and Spearman correlation.

Code and scripts are available in my UiO GitHub repository: <https://github.com/noorfa/IN5550-Oblig1>

Dataset and Regression Task

Dataset path on Fox: `/fp/projects01/ec403/IN5550/obligatories/1/emo_dataset.tsv.gz`

This assignment is a supervised text regression task.

Given an input text 'x', the model predicts a continuous valence score 'y'.

Prediction target: 'normalized' (min-max normalized valence in $[0,1]$)

Since it is continuous and bounded in $[0,1]$, which is suitable for regression and supports stable metric comparison across experiments.

Train/dev split

Script used:

- `'split_data.py'`

Procedure:

1. Load full dataset from Fox.
2. Set fixed seed ('42') for reproducibility.
3. Convert continuous 'normalized' labels into quantile bins using `'pd.qcut'`.
4. Perform stratified `'train_test_split'` ('80/20') with these bins.
5. Drop helper bin column and save split files.
6. Validate split quality by checking:
 - mean/std/min/max of 'normalized'
 - max difference in quantile-bin proportions between train and dev

Saved files:

- `'data/train.tsv.gz'`

- `'data/dev.tsv.gz'`

Final sizes:

- Train: '6840'
- Dev: '1711'
- Total: '8551'

Why this matters:

- Fixed split ensures fair comparison across model variants.
- The same split is reused in all Part 1 and Part 2 experiments.
- Stratification helps keep label distribution similar between subsets.
- This reduces split bias and makes dev metrics more reliable.

Train/Dev Label Distribution Check:

Train	mean=0.523229	std=0.102787	min=0.029412	max=1.000000
Dev	mean=0.522161	std=0.100532	min=0.000000	max=0.882353

Distribution sanity rule used in code:

- Mean difference threshold: < 0.01
- Max quantile-bin proportion difference threshold: < 0.03.

Mean difference: 0.0011

Max bin proportion difference: 0.0140

Train and dev normalized statistics are similar, confirming the split quality.

Part 1: Bag-of-Words Baseline

In Part 1, I built a Bag-of-Words (BoW) baseline for supervised valence regression.

The goal is to convert text into numeric BoW features and train a regression MLP to predict normalized valence in [0,1].

A) Code Explanation:

Code in bow.py (step-by-step, with what each part does):

1. **Load fixed splits :**
The script reads train.tsv.gz and dev.tsv.gz with pandas.

Using fixed splits means all model variants are compared on exactly the same data.
2. **Choose input column (data_col):**
The code can use:
text (raw text), lemma (lemmatized text), PoS (lemma + POS tags)
3. **Create vectorizer (make_vectorizer):** The code builds:

CountVectorizer for count and binary,

TfidfVectorizer for tfidf.

This defines how text becomes BoW numbers.

4. **Infer vocabulary (fit):**

`vectorizer.fit(...)` learns the token-to-index mapping. The default setting is `--vocab_scope train` (learn from train only), which avoids dev leakage.

5. **Build BoW features (transform):**

`vectorizer.transform(...)` converts each document into a sparse BoW vector.

Resulting shapes are:

train: [N_train, V]

dev: [N_dev, V]

where N is the number of documents and V is vocabulary size (limited by `max_features`).

6. **Extract gold valence labels:**

Targets are read from the normalized column:

```
y_train = train_df["normalized"], y_dev = dev_df["normalized"]
```

So label *i* matches document *i*.

7. **Prepare data for PyTorch**

BoW features and labels are converted to float32 tensors. Then they are wrapped in `TensorDataset` and `DataLoader` for mini-batch training.

8. **Build regression model (make_model)**

The model is an MLP:

input size = V,

output size = 1 (regression score),

hidden depth controlled by `n_layers`,

ReLU + dropout used in deeper models.

9. **Train model**

Loss: `MSELoss`

Optimizer: `AdamW`

Model output is passed through sigmoid, so predictions stay in [0,1].

10. **Select best checkpoint**

After each epoch, dev metrics are computed. The script keeps the checkpoint with lowest dev MSE and restores it at the end.

11. **Final evaluation and saving**

Final Train/Dev metrics are reported:

MSE, MAE, RMSE, Pearson, Spearman.

- Then the script saves a .pt file containing: model weights, fitted vectorizer, run config/args, with compatibility fields for eval_on_test.py.

B) Hyperparameters used

- max_features=5000
- hidden_size=256
- n_layers $\in \{1,2,3,4\}$
- dropout=0.2
- epochs=25
- batch_size=64
- lr=1e-3 (learning rate)
- seed=42 (grid), then seeds 42/43/44 for best-config repeat.

C) Experiments

I ran 10 controlled experiments in run_bow_experiments.sh.

I varied:

- BoW feature type: count, binary, tfidf
- Input text column: lemma, text, PoS
- Number of MLP layers: 1–4

Hypotheses before running:

- More layers should improve results (at least up to some depth).
- count should be stronger than binary and tfidf for this task.
- lemma should work better than raw text and PoS.

Results :

bow_metrics_summary.csv X +											
Delimiter: ,											
	run	train_rmse	train_mae	train_rmse	train_pearson	train_spearman	dev_rmse	dev_mae	dev_rmse	dev_pearson	dev_spearman
1	exp04_lemma_count_L4	0.000312	0.011974	0.017674	0.987535	0.981336	0.007205	0.061554	0.064885	0.555594	0.525162
2	exp03_lemma_count_L3	0.00031	0.01112	0.017593	0.98528	0.979276	0.007245	0.061308	0.085119	0.554154	0.521067
3	exp02_lemma_count_L2	0.000386	0.012027	0.019639	0.981649	0.974623	0.007674	0.063484	0.087602	0.533292	0.502822
4	exp05_lemma_binary_L2	0.000412	0.012146	0.020296	0.980591	0.974719	0.007758	0.064371	0.08808	0.523912	0.484128
5	exp09_pos_count_L2	0.00044	0.012584	0.02097	0.979932	0.973842	0.007998	0.065285	0.08943	0.502717	0.472136
6	exp06_lemma_tfidf_L2	0.000295	0.010865	0.017178	0.985966	0.976444	0.008322	0.067163	0.091223	0.518319	0.495933
7	exp08_text_binary_L2	0.000466	0.012296	0.021595	0.977839	0.973017	0.008375	0.06667	0.091516	0.468366	0.4371
8	exp07_text_count_L2	0.000458	0.01243	0.02141	0.978377	0.972857	0.008473	0.066601	0.092048	0.459331	0.438943
9	exp01_lemma_count_L1	0.003538	0.039195	0.059485	0.822888	0.838157	0.008698	0.06878	0.093263	0.47107	0.474821
10	exp10_pos_tfidf_L2	0.000351	0.011662	0.018734	0.98423	0.975467	0.008878	0.069389	0.094225	0.485383	0.449953

Best run:

exp04_lemma_count_L4

Lowest dev MSE: 0.007205

Highest dev Pearson: 0.555594

Highest dev Spearman: 0.525162

Hypothesis vs outcome

H1 (more layers help): supported. L1 was weakest; L3/L4 were strongest.

H2 (count > binary/tfidf): supported. Count consistently performed better in matched settings.

H3 (lemma > text/PoS): supported. Best runs came from lemma-based features.

Best config repeated 3 times (required)

I retrained the best Part 1 setup with seeds 42, 43, 44.

Per-run dev MSE:

s42: 0.007205

s43: 0.007252

s44: 0.007217

Aggregate (mean $\pm$ std):

```
Launcher x bow_metrics_aggreg: x
```

```
1 ## Aggregate Over Selected Runs
2
3 Runs: 3
4
5 | Metric | Mean | Std |
6 |----|---:|---:|
7 | Dev MSE | 0.007225 | 0.000024 |
8 | Dev MAE | 0.061291 | 0.000515 |
9 | Dev RMSE | 0.084999 | 0.000144 |
10 | Dev PEARSON | 0.553609 | 0.002732 |
11 | Dev SPEARMAN | 0.523774 | 0.001381 |
12 |
```

This shows stable performance across seeds (very low std).

D) Conclusion for Part 1:

The strongest BoW baseline in this project is:

lemma input, count features, 4-layer MLP

This model is used as the Part 1 reference baseline for comparison with Part 2 embedding models.

Part 2: Word Embeddings Baseline

In Part 2, I changed text representation from sparse BoW vectors to dense word embeddings.

A) Code explanation

Main files used:

- embeddings.py
- datasets.py (EmbeddingsDataset)
- data_loading.py (CollateFuncutor)
- model.py (EmbeddingModel)
- Load_embedding.py
- train_own_embeddings.py (for own-embedding experiment)

1. Load fixed train/dev data

The script reads [train.tsv.gz](#) and [dev.tsv.gz](#). (same as part 1)

2. Load pre-trained embeddings and add special tokens [UNK] and [PAD]

Then loads pretrained embedding using word2vec.

And then using ensure_special_tokens(word2vec) makes sure both are available:

[UNK] vector = average of all embedding vectors, [PAD] vector = zero vector

This is needed because:

- Unknown tokens must map somewhere ([UNK])
- Batch padding needs a dedicated index ([PAD])

5. Convert text to token-id sequences (EmbeddingsDataset)

EmbeddingsDataset takes a dataframe row and:

- reads selected text column (args.data_col)
- tokenizes text
- maps each token to embedding index

6. Build padded mini-batches (CollateFuncutor)

CollateFuncutor(pad_idx, max_length) is passed to DataLoader.

For each batch it collect variable-length sequences, truncate long ones to max_length and pad short ones with pad_idx

This is the key step that turns variable-length text into fixed-size tensors that a feed-forward model can process.

7. Build embedding regression model (EmbeddingModel)

EmbeddingModel(args, word2vec, n_labels=1) contains:

1. Embedding layer:
 - initialized from pre-trained matrix (word2vec.vectors)
 - can be frozen or trainable (--freeze_embeddings)
 2. Pooling layer (sentence representation):
 - input token embeddings shape: [B, L, D]
 - output sentence vector shape: [B, D]
 - pooling options:
 - mean: average token vectors
 - sum: sum token vectors
 - max: elementwise max token value
 3. MLP head:
 - maps [B, D] to [B, 1] regression output
- So the full path is:
token_ids -> token vectors -> pooled sentence vector -> MLP -> valence prediction

8. Train loop

Core training loop in embeddings.py:

- for input_ids, labels in train_loader: reads one mini-batch.
- input_ids = input_ids.to(device) and labels = labels.to(device)
move tensors to CPU/GPU device.
- optimizer.zero_grad() : clears old gradients.
- preds = model(input_ids).squeeze(-1) : forward pass.
- loss = criterion(preds, labels) with criterion = nn.MSELoss(): computes regression loss.
- loss.backward(): computes gradients.
- optimizer.step(): updates model weights.

After each epoch:

- evaluate on dev set
- print epoch train/dev MSE
- if dev MSE improves, save checkpoint state

At end: restore best checkpoint (lowest dev MSE).

9. Evaluate metrics

evaluate() collects all predictions and labels, then computes: MSE, MAE, RMSE, Pearson correlation, Spearman correlation

10. Save model in eval-compatible format

The final .pt file stores model weights, run arguments and compatibility keys expected by eval_on_test.py

B) Hyperparameters used

- data_col=PoS
- max_length=128
- batch_size=64
- hidden_size=256
- n_layers=2
- dropout=0.2
- epochs=8
- lr=1e-3
- seed 42 for grid, then seeds 42/43/44 for best-config repeat

C) What I varied and why (experiments + hypotheses)

I varied:

- pooling: mean, sum, max
- OOV handling: unk vs pad
- embedding update mode: frozen vs fine-tuned
- pre-trained source model: static 200 vs static 223
- pre-trained vs own-trained embeddings

Hypotheses:

1. Mean pooling will be a strong baseline.
2. Fine-tuning embeddings should outperform freezing.
3. OOV strategy can affect quality.
4. Different pre-trained models can produce different results.
5. Pre-trained large vectors may outperform own vectors trained on smaller corpus.

6. Embedding algorithm family (word2vec vs fastText) can change quality.

Algorithm-Family Comparison (word2vec vs fastText)

Additionally, I ran one controlled algorithm-family comparison between a word2vec model and a fastText model, using the same setup (text + lowercase, mean pooling, unk OOV, fine-tuned embeddings, same seed/epochs). This isolates the effect of embedding algorithm type. The run summary is available in bow_metrics_summary.md.”

D) Pre-trained runs (results)

	run	train_mse	train_mae	train_rmse	train_pearson	train_spearman	dev_mse	dev_mae	dev_rmse	dev_pearson	dev_spearman
1	200_mean_pad_tune_e8	0.003825	0.042846	0.061846	0.810795	0.806166	0.006659	0.061053	0.081604	0.591558	0.563942
2	200_sum_unk_tune_e8	0.003104	0.037102	0.055716	0.851781	0.849661	0.006687	0.060424	0.081772	0.585612	0.541802
3	223_mean_unk_tune_e8	0.004783	0.048359	0.069161	0.753195	0.763195	0.006795	0.061841	0.082431	0.580714	0.550389
4	200_mean_unk_tune_e8	0.00437	0.046182	0.066103	0.781574	0.790357	0.006857	0.061939	0.082806	0.580396	0.562895
5	00_mean_unk_freeze_e8	0.00632	0.056946	0.079501	0.639456	0.612077	0.006921	0.060708	0.083191	0.561838	0.536969
6	200_max_unk_tune_e8	0.002574	0.034063	0.050737	0.882282	0.886969	0.00701	0.062974	0.083723	0.565767	0.519885
7	text_mean_unk_tune_e8	0.006497	0.056244	0.080601	0.645712	0.681776	0.008308	0.068018	0.091146	0.446573	0.443769
8	txt_mean_unk_freeze_e8	0.00824	0.063796	0.090773	0.474652	0.480963	0.008439	0.066309	0.091865	0.406363	0.409277

Algorithm comparison result

	run	train_mse	train_mae	train_rmse	train_pearson	train_spearman	dev_mse	dev_mae	dev_rmse	dev_pearson	dev_spearman
1	text_mean_unk_tune_e8	0.005363	0.049225	0.07323	0.712184	0.738915	0.00757	0.063308	0.087005	0.508121	0.495834
2	text_mean_unk_tune_e8	0.010565	0.072093	0.102787	nan	nan	0.010101	0.071466	0.100503	nan	nan

Best run: Emb_200_mean_pad_tune_e8

E) Hypothesis vs Outcome

H1 (mean pooling > sum/max): supported. Mean-pooling setup gave the best overall Part 2 run.
H2 (fine-tuning > freezing): supported. Fine-tuned embedding runs performed better than frozen runs in matched settings.
H3 (OOV strategy matters): supported. pad and unk gave different outcomes; best run used pad with mean pooling.
H4 (different pre-trained sources change results): supported. Static 200 and static 223 produced different dev performance under comparable settings.
H5 (pre-trained > own-trained embeddings): supported. Best pre-trained run clearly outperformed best own-embedding run.
H6 (algorithm family matters: word2vec vs fastText): supported. In controlled comparison, fastText outperformed word2vec (0.007570 vs 0.010101 dev MSE).

F) Best configuration repeated 3 times (required)

Best config rerun with seeds 42, 43, 44.

Aggregate (mean ± std):

```
| Metric | Mean | Std |
|---|---:|---:|
| Dev MSE | 0.006760 | 0.000137 |
| Dev MAE | 0.061219 | 0.000487 |
| Dev RMSE | 0.082215 | 0.000832 |
| Dev PEARSON | 0.590122 | 0.001250 |
| Dev SPEARMAN | 0.560427 | 0.003257 |
```

This shows low variance and good stability.

G) Own embeddings

I trained my own Word2Vec embeddings on at least 10M English tokens from:

- /fp/projects01/ec403/corpora

Why this corpus:

I chose it for four reasons.

- First, the language matches the task data (English), so learned word relations are relevant.
- Second, it satisfies the assignment size requirement (≥ 10 M tokens), which is important for stable embedding quality.
- Third, the corpus is large and diverse enough to improve vocabulary coverage and reduce out-of-vocabulary issues on train/dev texts.
- Fourth, it is available on Fox, so experiments are easy to reproduce.

A limitation is that this is a general-domain corpus, not emotion-specific text, which may explain why pre-trained large models still performed better in my final comparison.

Training setup (best own embedding model):

- skip-gram (sg=1)
- vector_size=300
- window=5
- min_count=5
- epochs=5

Saved files: model.gensim, model.bin, model.vec

Own-embedding regression results:

- emb_own_best_text_mean_unk_tune_e8: Dev MSE 0.008308
- emb_own_best_text_mean_unk_freeze_e8: Dev MSE 0.008439

H) Conclusion for Part 2

Best Part 2 model uses:

- pre-trained static 200
- PoS input
- mean pooling
- oov_strategy=pad
- fine-tuned embeddings
- 2-layer MLP

Comparison:

- best pre-trained Dev MSE: 0.006659
- best own-embedding Dev MSE: 0.008308

So **pre-trained embeddings** perform better than my own embeddings on this task.



Evaluation

Evaluation was done with the official script.

This verifies that the saved model format is compatible with the course evaluation pipeline.

For this run, the script reported:

MSE 0.0073, MAE 0.0622, RMSE 0.0854, Pearson 0.5788, Spearman 0.5653.